Email: John.dixon@hull.ac.uk
Meetings: https://tinyurl.com/jdixonhull
Feedback: https://tinyurl.com/jdhullfeedback
Office: RBB-315

Concurrency Principles for Distributed Computing

Distributed Systems Programming

Dr John Dixon

UNIVERSITY of HULL

Overview

We are looking at these topics today:

- Overcoming the problems with concurrency through the use of different mechanisms, e.g.
  - Locks and variants of lock
  - Deadlocks
  - Optimistic Concurrency Control (OCC)
  - Timestamping
- And implementation details:
  - Locks in C#
  - Mutex in C#
  - Semaphore in C#

UNIVERSITY of HULL

Recap

- Concurrency is key in distributed applications so that:
  - Many clients can simultaneously access a single endpoint (server)
  - Clients can simultaneously use many servers
  - Applications can make maximum simultaneous use of distributed resources
  - Applications can make maximum simultaneous use of distributed data
- However, simultaneous access can result in some severe errors
- Therefore we have to prevent these from happening
- In this lecture we'll look at methods of doing this for distributed applications; especially those developed in C# / .NET

UNIVERSITY of HULL

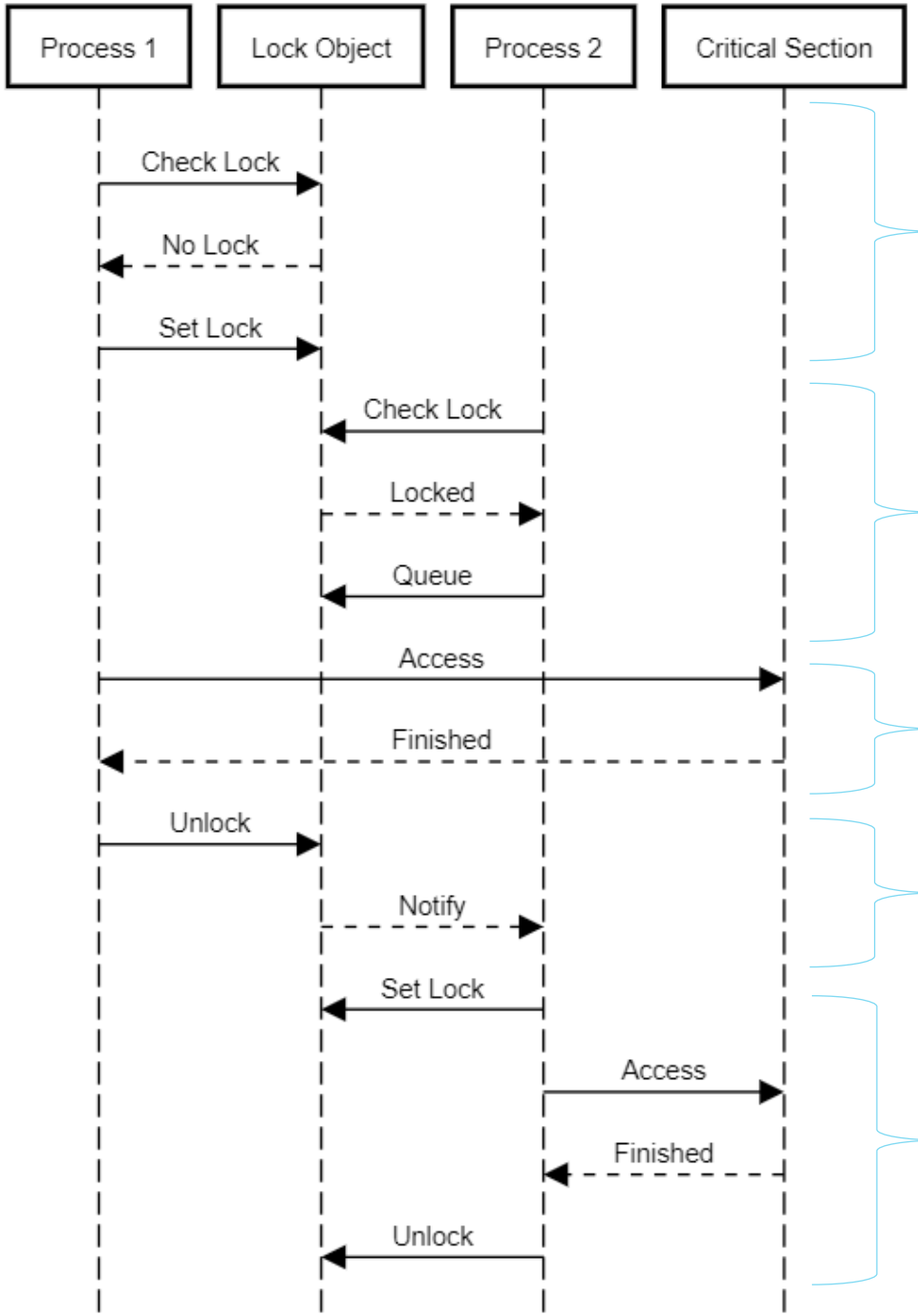
Locks

- We have a ***Critical Section*** requiring ***Mutual Exclusion*** to ensure synchronisation of an operation than can only safely be performed if another ***Thread*** is not currently in the ***Critical Section***.
- We can employ ***Locks*** to act as our guard.

- A lock can be placed to guard any object
- As a transaction accesses the object, the lock is set and no other transaction may use the object.
- Once the accessing transaction has finished using the object the lock is removed.

- Locks may only be placed using Atomic Operations (only one process may be active whilst a lock is being placed to avoid two processes trying to simultaneously lock)

UNIVERSITY of HULL



Process 1 queries lock
No lock so sets one before accessing critical section

Process 2 queries lock
Lock is set so must now wait (queue could be implemented in Lock or Process 2 could back off and retry later

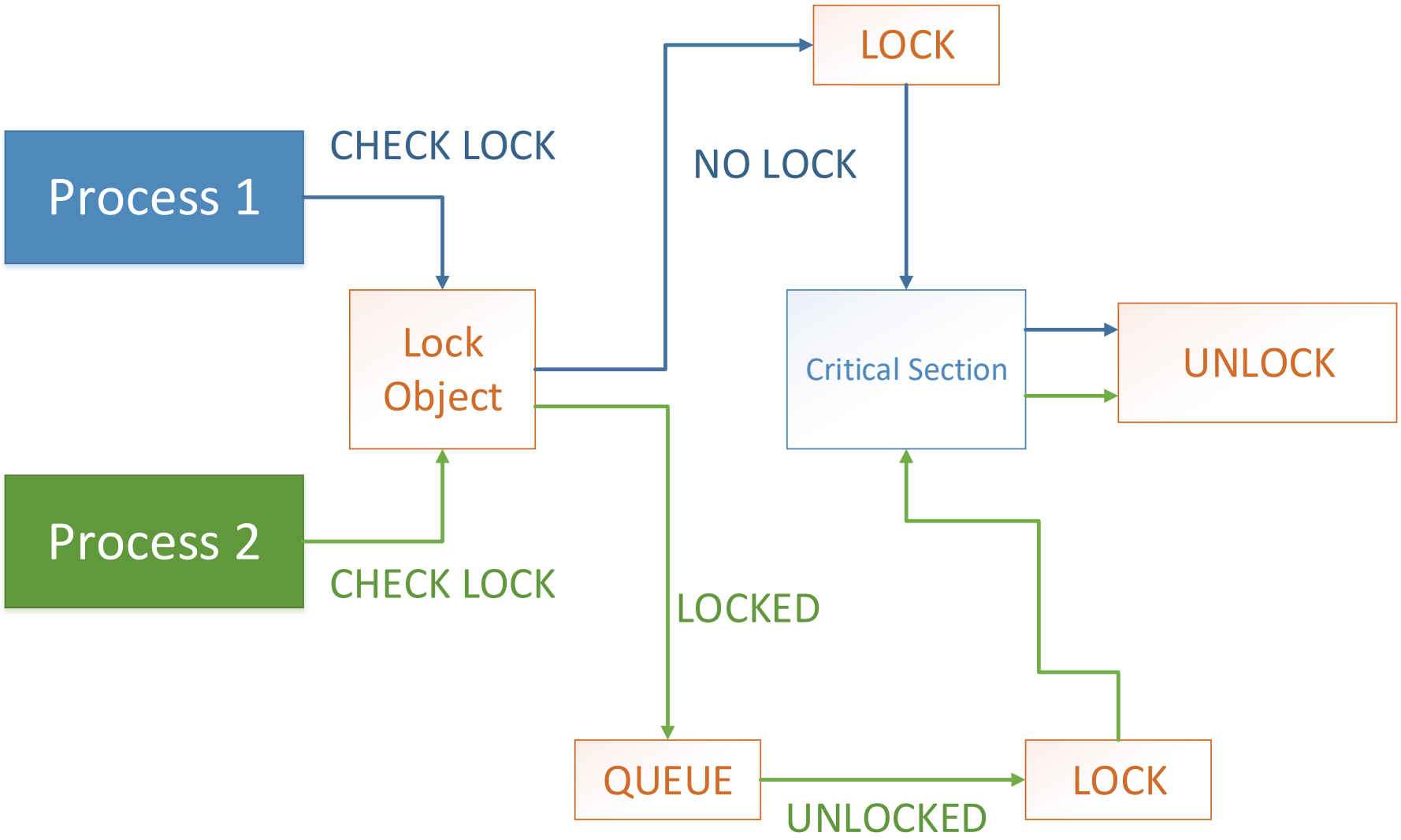
Process 1 uses Critical Section

Process 1 releases the Lock
Lock may notify Process 2

Process 2 sets Lock, uses Critical Section and unlocks when finished

UNIVERSITY of HULL

Or…if you prefer…



UNIVERSITY of HULL

Deadlock

A transfer of £100 is required from account *a* to account *b*. A separate process tries to calculate the total funds in all accounts.

Time

Process 1 (transfer £100 from a to b)

Operation	temp	Balance a	Balance b
Read a		£300	£500
Sub £100	£200	£300	£500
Write a	£200	£200	£500
Read b	£500	£200	b Locked

1 Locks a

Waiting for *b* to be unlocked by 2

Process 2 (total accounts)

Operation	Total	temp
Read b		£500
Add temp to total	£500	£500
Read a	a Locked	£300

2 Locks b

Waiting for *a* to be unlocked by 1

Deadlock

UNIVERSITY of HULL

Deadlocks

- A deadlock may be caused by a long chain of dependencies or a very complex cycle.
- Sometimes very hard to predict

- Resolution:
  - At least one transaction must be aborted

- Solutions?
  - Lock all resources required by a transaction as it starts (may be very restrictive on a big system)
  - Detect deadlocks by looking for cycles when locks are granted (overhead and which to abort?)
  - Enforce timeout (may mean long-running transactions are incorrectly aborted)

Example: Locking in C#

```
private object myLock = new object();

int Withdraw(int amount)
{
    lock (myLock)
    {
        if (balance >= amount)
        {
            Console.WriteLine("Balance before Withdrawal : " + balance);
            Console.WriteLine("Amount to Withdraw : -" + amount);
            balance = balance - amount;
            Console.WriteLine("Balance after Withdrawal : " + balance);
            return amount;
        }
        else
        {
            return 0; // transaction rejected
        }
    }
}
```

UNIVERSITY of HULL

Locking – Best Practices

- Any object can be used as the lock
- Avoid locking using a **public** type, or instances beyond your code's control.
  - Another developer may be locking on a public type, in any code (esp. if you are using a library or your code is a library)
- Some common constructs violate this guideline:
  - `lock(this)` is a problem if the instance can be accessed publicly.
  - `lock(typeof(MyType))` is a problem if MyType is publicly accessible.
  - `lock("myLock")` is a problem since any other code in the process using the same string will share the same lock.
- Best practice is to define a **private** object to lock on, or a shared **private static** object variable to protect data common to all instances.

https://docs.microsoft.com/en-us/dotnet/csharp/language-reference/keywords/lock-statement

UNIVERSITY of HULL

Example: Mutex in C#

- A C# Mutex is a lock on a **system scale** – it allows concurrency control across multiple linked applications

```
Mutex m = new Mutex();

try
{
   // Try to open existing system mutex.
   m = Mutex.OpenExisting("myMutex"); // Best to use a unique namespace...

}
catch
{
   // If exception occurred, there is no such mutex.
   m = new Mutex(true, "myMutex"); // Make mutex
}

// Get the Mutex (or block until available)
m.WaitOne();

// Do some work.

// Release the Mutex.
m.ReleaseMutex();
```

UNIVERSITY of HULL

Example: Semaphore in C#

- A C# Semaphore allows only a <u>given number</u> of threads to access a resource simultaneously
- In theory, a semaphore could act like a conventional lock if the number of slots was set to 1
- Semaphores help prevent overloading or thin resource availability
- Semaphore(initial, max) -- if initial < max, extra slots must be released before use

```
Semaphore s = new Semaphore(10, 10);

// Get a semaphore slot
s.WaitOne();

// Do some work

// Release the semaphore slot
s.Release();
```

- In similarity with a Mutex, a semaphore can be applied system-wide if it is named:

```
Semaphore s = new Semaphore(10, 10, "mySemaphore");
```

UNIVERSITY of HULL

Locking – Disadvantages

- Locking (and Mutex/Semaphores) is essential to preventing conflicting updates but can adversely affect performance

- If scope of lock is too great (extreme example: locks entire database)
  - forces some threads to wait
  - decreases potential concurrency and throughput / performance

- If scope of lock is too small (extreme example: one variable at a time)
  - forces reacquisition of locks, leading increased lock contention and reduced throughput / performance
  - can result in difficult-to-debug race conditions
  - Higher chances of deadlock

ReaderWriter Locking

We've previously determined that some transactions are inherently safe when they are conducted concurrently because they are ***serially equivalent***

- E.g. Concurrent READ operations

We can therefore manually use a different lock object when reading (vs. when writing)

OR we can use a…

- ReaderWriter lock
  - Allows concurrent read access for multiple threads, **or**
  - Write access for a single thread
  - Uses separate locks for readers and writers
  - Clients must specify which lock they want

UNIVERSITY of HULL

ReaderWriter Locking

- Reads are concurrent, writes are not
- Locks can be upgraded (so a thread can request READ then later request WRITE only around the writeback)

		Lock requested	
		Read	Write
Current lock	None	Grant	Grant
	Read	Grant	Wait
	Write	Wait	Wait

How can a deadlock occur?

Process A and B have READ. Both processes request upgrade to WRITE

Neither can upgrade because the other has read.

UNIVERSITY of HULL

Advanced locking: Reader lock

```
ReaderWriterLock rwl = new ReaderWriterLock();

void ReadFromResource(int timeOut)
{
    try
    {
        rwl.AcquireReaderLock(timeOut);
        try
        {
            // Safe for this thread to read from the shared resource.
        }
        finally
        {
            // Ensure that the lock is released.
            rwl.ReleaseReaderLock();
        }
    }
    catch (ApplicationException)
    {
        // The reader request timed out.
    }
}
```

UNIVERSITY of HULL

Advanced locking: Writer lock

```
void WriteToResource(int timeOut)
{
    try
    {
        rwl.AcquireWriterLock(timeOut);
        try
        {
            // It is safe for this thread to read from or write
            // to the shared resource.
        }
        finally
        {
            // Ensure that the lock is released.
            rwl.ReleaseWriterLock();
        }
    }
    catch (ApplicationException)
    {
        // The writer lock request timed out.
    }
}
```

UNIVERSITY of HULL

Two-Version Locking

- With this type of lock write operations are made on temporary copies of data (e.g. in memory). Commits actually make the change in the data store.

- Uses read, write and commit locks
  - **Commit locks** are required to **save** data to the permanent store
  - **Read locks** granted on objects which do not have commit locks
  - **Write locks** granted on objects which do not have write or commit locks (one write at a time)

- When ready to commit, a write lock must be promoted to a commit lock
  - A commit lock is only granted when **all** other locks are released on the object

- Reads are delayed as little as possible. Write is granted whilst other processes have read locks
- Commits may be delayed by read, but reads are usually short
- Commits can 'fire when ready'

UNIVERSITY of HULL

Two-Version Locking

		Lock requested		
		Read	Write	Commit
Current lock on Object	None	Grant	Grant	Grant
	Read	Grant	Grant	Wait
	Write	Grant	Wait	
	Commit	Wait	Wait	

Still a possibility of a deadlock where two processes Read, then Write, then Commit:

P1 Read → P2 Read → P1 Write → P2 Write(wait) → P1 Commit(wait)

P2 should back off and re-read after P1's commit

UNIVERSITY of HULL

Hierarchical Locking

- Using hierarchical locking, locks exist at different levels of granularity
  - Whole bank
    - Specific branch
      - Individual account in a branch
        - Individual function on an account
- Scope of locks should be just right for the task, neither too great nor too small
  - Improves concurrency
  - Each task should know which level of the hierarchy to lock to perform its function
- Simplifies granting of locks on large collections of resources
  - e.g. if calculating branch totals – lock the whole branch
  - e.g. if moving money between accounts – only lock relevant accounts

Locking – Benefits and Drawbacks

- Locking will **prevent** data read/write issues
- Locks are easy to implement
- There are different types of lock for different situations

But…

- Locking is **pessimistic**. It assumes a clash will occur so introduces delays to prevent this
  - Clashes don't always occur (in fact they are quite rare in many systems)
- Locks may create overhead – even on reads
  - Not necessary except in worst case
- Locks can create deadlock
  - More overhead in detection or timeout
- Locks are only released at end of transaction
  - Decreases concurrency

Optimistic Concurrency Control (OCC)

- OCC works on the assumption that the likelihood of concurrent access to an object is actually rather low in many situations

- Transactions proceed as if conflict impossible
- At end of transaction, conflicts are checked for
- If conflict occurs, one or more transactions are aborted

Optimistic Concurrency Control (OCC)

- Three stages:

- Validation phase:
  - Checks to see if conflicts with overlapping transactions have occurred
    - Backward validation: transaction may not read objects written by transactions that validated while this transaction was working
    - Forward validation: transaction may not write objects read by transactions in working phase
  - If so, no changes are made (abort/loop back and retry)
    - Backward validation: abort transaction being validated
    - Forward validation: abort either validating or working transaction
  - If not, all changes may be committed

UNIVERSITY of HULL

OCC in EF

The workshop investigates optimistic concurrency control for Entity Framework

In order to achieve OCC, EF uses row versioning

- Each row in the table is versioned using a timestamp
- The timestamp is a column added to the table (and to the data entity)
- This ensures no overlaps have occurred, but only when a save is attempted

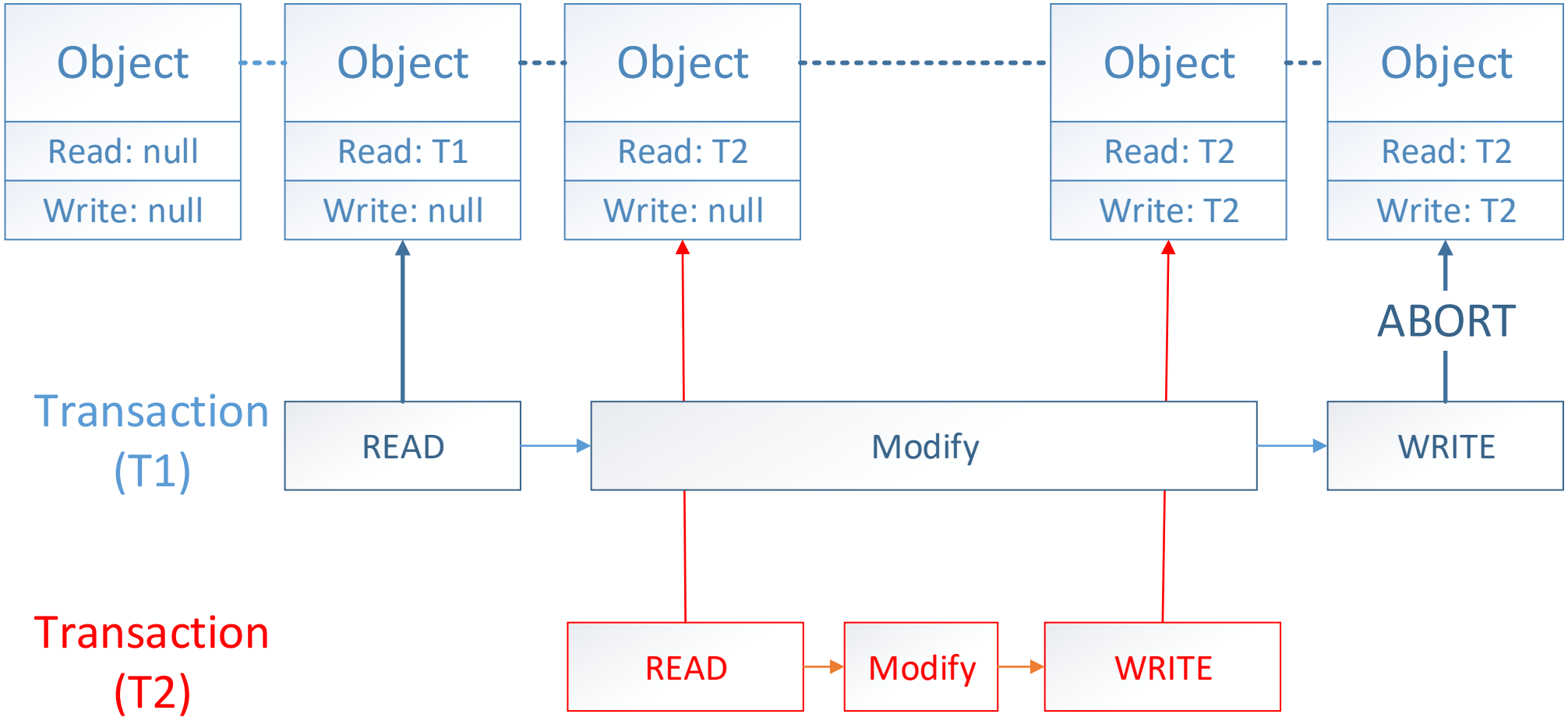
- Entity Framework manages this for us once we put the building blocks in place
- We must manage what to do if a concurrency error occurs

UNIVERSITY of HULL

Timestamping

- Each *transaction* is allocated a timestamp
- Each *object* has a read and a write timestamp
  - Read timestamp is of last committed transaction that read the object
  - Write timestamp is last committed transaction that changed the object
- Each operation is validated as it is carried out
  - Write requests are only valid if the object was last read and written by earlier transactions
  - Read requests are only valid if the object was last written by an earlier transaction
  - Aborted if not valid

Timestamping



T1 is aborted as T2 has modified the timestamp since T1 did a read
T1 likely restarts from a new read and has to do its work again

UNIVERSITY of HULL

Timestamping & OCC

- No deadlocks
- No overhead in setting/checking locks or queuing processes
- Non-clashing synchronous tasks will go ahead when they would otherwise have to wait
- Great if you expect few clashes as you can minimise delays that would be caused by unnecessary locks

But…

- Some (small) overhead in checking for any changed data before a save
- If a clash is found, affected processes must completely restart
- Long-running processes may be penalised
- Not good if you expect lots of clashes as you will introduce more delays through restarts

Additional Locking Mechanisms

These are the main concepts.

There are a LOT of other locking and synchronisation control mechanisms in .NET

Start your additional reading here…

https://docs.microsoft.com/en-us/dotnet/api/system.threading

Some key areas for concurrency that we haven't gone into in detail are:

- Monitors
- Wait Handles
- Barriers

And in Entity Framework:

- Concurrency: https://docs.microsoft.com/en-us/ef/core/saving/concurrency
- and Atomic Transaction Control: https://docs.microsoft.com/en-us/ef/core/saving/transactions

UNIVERSITY of HULL

Reflection

In this session we've looked at:

- Locks and variants of lock (Locks in C#, Mutex in C#, Semaphore in C#)
- Deadlocks
- Optimistic Concurrency Control (OCC)
- Timestamping

In the next session we will be looking at design and deployment of Distributed Computing Solutions

UNIVERSITY of HULL